

Лабораторная работа №2

```
6  -- 1. СТОЛОВЫЕ (Справочник точек питания)
7  CREATE TABLE dining_room (
8      id SERIAL PRIMARY KEY,
9      name VARCHAR(100) NOT NULL,
10     address VARCHAR(200) NOT NULL,
11     capacity INT NOT NULL CHECK (capacity > 0),
12     is_open BOOLEAN DEFAULT true,
13     open_time TIME NOT NULL,
14     close_time TIME NOT NULL,
15     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
16     CONSTRAINT unique_dining_name_address UNIQUE (name, address)
17 );
18
19 -- 2. СОТРУДНИКИ
20 CREATE TABLE employee (
21     id SERIAL PRIMARY KEY,
22     dining_room_id INT NOT NULL,
23     first_name VARCHAR(50) NOT NULL,
24     last_name VARCHAR(50) NOT NULL,
25     position VARCHAR(50) DEFAULT 'Повар',
26     hire_date DATE NOT NULL DEFAULT CURRENT_DATE,
27     salary NUMERIC(10, 2) CHECK (salary > 0),
28     phone VARCHAR(20) UNIQUE,
29     email VARCHAR(100) UNIQUE,
30     CONSTRAINT fk_employee_dining_room FOREIGN KEY (dining_room_id) REFERENCES di
31
32
33 -- 3. ПОСТАВЩИКИ
34 CREATE TABLE supplier (
35     id SERIAL PRIMARY KEY,
36     company_name VARCHAR(150) NOT NULL,
37     contact_person VARCHAR(100),
38     phone VARCHAR(20) NOT NULL,
39     email VARCHAR(100),
40     address VARCHAR(250),
41     rating INT DEFAULT 5 CHECK (rating >= 1 AND rating <= 10),
42     contract_start DATE DEFAULT CURRENT_DATE
43 );
44
45 -- 4. ПРОДУКТЫ
46 CREATE TABLE product (
47     id SERIAL PRIMARY KEY,
48     supplier_id INT NOT NULL,
49     name VARCHAR(100) NOT NULL,
50     unit VARCHAR(20) NOT NULL DEFAULT 'кг', -- кг, л, шт
51     purchase_price NUMERIC(8, 2) NOT NULL CHECK (purchase_price > 0),
52     calories_per_unit INT CHECK (calories_per_unit >= 0),
53     expiration_days INT DEFAULT 7,
54     is_alcohol BOOLEAN DEFAULT false,
55     CONSTRAINT fk_product_supplier FOREIGN KEY (supplier_id) REFERENCES supplier(
56 )
```

```

-- 5. БЛЮДА МЕНЮ
CREATE TABLE menu_item (
    id SERIAL PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    category VARCHAR(50) NOT NULL CHECK (category IN ('Суп', 'Горячее', 'Салат',
selling_price NUMERIC(8, 2) NOT NULL CHECK (selling_price > 0),
weight_grams INT DEFAULT 200,
is_available BOOLEAN DEFAULT true,
description TEXT
);

-- 6. СОСТАВ БЛЮДА (М:Н связь)
CREATE TABLE menu_item_product (
    id SERIAL PRIMARY KEY,
    menu_item_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity NUMERIC(7, 3) NOT NULL CHECK (quantity > 0), -- Количество продукта
CONSTRAINT fk_mip_menu FOREIGN KEY (menu_item_id) REFERENCES menu_item(id) ON
CONSTRAINT fk_mip_product FOREIGN KEY (product_id) REFERENCES product(id) ON
CONSTRAINT unique_menu_product UNIQUE (menu_item_id, product_id)
);

-- 7. КЛИЕНТЫ (Посетители)
CREATE TABLE client (
    id SERIAL PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    phone VARCHAR(20) UNIQUE,
    birthday DATE,
    discount_card VARCHAR(20) UNIQUE,
    discount_percent INT DEFAULT 0 CHECK (discount_percent >= 0 AND discount_perc
registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 8. ЗАКАЗЫ
CREATE TABLE meal_order (
    id SERIAL PRIMARY KEY,
    client_id INT, -- может быть NULL, если гость без карты
    employee_id INT, -- кассир/официант
    dining_room_id INT NOT NULL,
    order_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    is_takeaway BOOLEAN DEFAULT false,
    total_amount NUMERIC(10, 2) DEFAULT 0,
    payment_method VARCHAR(20) CHECK (payment_method IN ('Наличные', 'Карта', 'QR
CONSTRAINT fk_order_client FOREIGN KEY (client_id) REFERENCES client(id) ON D
CONSTRAINT fk_order_employee FOREIGN KEY (employee_id) REFERENCES employee(id
CONSTRAINT fk_order_dining FOREIGN KEY (dining_room_id) REFERENCES dining_roo

```

```

-- 9. ПОЗИЦИИ В ЗАКАЗЕ
CREATE TABLE order_item (
    id SERIAL PRIMARY KEY,
    meal_order_id INT NOT NULL,
    menu_item_id INT NOT NULL,
    quantity INT NOT NULL CHECK (quantity > 0),
    price_per_unit NUMERIC(8, 2) NOT NULL, -- Цена на момент заказа
    CONSTRAINT fk_oi_order FOREIGN KEY (meal_order_id) REFERENCES meal_order(id)
    CONSTRAINT fk_oi_menu FOREIGN KEY (menu_item_id) REFERENCES menu_item(id) ON
);

-- 10. СКЛАДСКИЕ ОСТАТКИ
CREATE TABLE inventory (
    id SERIAL PRIMARY KEY,
    dining_room_id INT NOT NULL,
    product_id INT NOT NULL,
    current_stock NUMERIC(10, 3) NOT NULL DEFAULT 0 CHECK (current_stock >= 0),
    min_stock_level NUMERIC(10, 3) DEFAULT 2.0,
    last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_inv_dining FOREIGN KEY (dining_room_id) REFERENCES dining_room(
    CONSTRAINT fk_inv_product FOREIGN KEY (product_id) REFERENCES product(id) ON
    CONSTRAINT unique_stock_per_dining UNIQUE (dining_room_id, product_id)
);

-- Столовые (5)
INSERT INTO dining_room (name, address, capacity, open_time, close_time) VALUES
('Столовая Центр', 'ул. Ленина, 10', 120, '08:00', '17:00'),
('Столовая Западная', 'пр. Мира, 22', 80, '09:00', '18:00'),
('Бизнес-ланч Сити', 'ул. Пушкина, 5', 45, '11:00', '19:00'),
('Студенческая', 'ул. Школьная, 3', 200, '08:30', '16:00'),
('Парковая трапеза', 'парк Культуры, 1', 60, '10:00', '20:00');

-- Поставщики (8)
INSERT INTO supplier (company_name, contact_person, phone, email, rating) VALUES
('ООО МясоТорг', 'Иванов И.И.', '89161112233', 'meat@mail.ru', 9),
('ЗАО Молоково', 'Петрова А.С.', '89162223344', 'milk@yandex.ru', 8),
('ИП Хлебный Дом', 'Сидоров В.В.', '89163334455', 'bread@mail.ru', 10),
('Фруктовый Сад ООО', 'Кукушкина О.Л.', '89164445566', 'fruit@list.ru', 7),
('Рыбный Мир', 'Сомов А.Ч.', '89165556677', 'fish@mail.ru', 8),
('Овощная База №1', 'Луков Н.П.', '89166667788', 'veggies@yandex.ru', 9),
('Напитки Плюс', 'Колов О.Е.', '89167778899', 'drinks@mail.ru', 8),
('Кондитерская Фея', 'Тортова Е.М.', '89168889900', 'cake@list.ru', 10);

-- Сетевые (12)

```

```

-- Сотрудники (12)
INSERT INTO employee (dining_room_id, first_name, last_name, position, salary, ph
(1, 'Алексей', 'Поваров', 'Шеф-повар', 85000, '8901111111'),
(1, 'Марина', 'Кашеварова', 'Повар', 55000, '89011111112'),
(1, 'Ольга', 'Чистюля', 'Посудомойка', 35000, '89011111113'),
(2, 'Иван', 'Жарков', 'Повар', 58000, '89012222221'),
(2, 'Светлана', 'Быстрова', 'Кассир', 45000, '89012222222'),
(3, 'Дмитрий', 'Салатов', 'Повар', 62000, '89013333331'),
(3, 'Наталья', 'Официантова', 'Официант', 40000, '89013333332'),
(4, 'Елена', 'Студенткина', 'Буфетчица', 38000, '89014444441'),
(4, 'Петр', 'Лапшин', 'Повар', 52000, '89014444442'),
(5, 'Андрей', 'Грильев', 'Шеф-повар', 88000, '89015555551'),
(5, 'Татьяна', 'Десертова', 'Кондитер', 60000, '89015555552'),
(1, 'Виктор', 'Закупкин', 'Кладовщик', 46000, '89011111114');

-- Продукты (15)
INSERT INTO product (supplier_id, name, unit, purchase_price, calories_per_unit,
(1, 'Говядина вырезка', 'кг', 650.00, 2500, 3),
(1, 'Куриное филе', 'кг', 280.00, 1100, 4),
(2, 'Молоко 3.2%', 'л', 65.00, 600, 5),
(2, 'Сметана 20%', 'кг', 220.00, 2050, 7),
(3, 'Хлеб ржаной', 'шт', 35.00, 2000, 2),
(3, 'Хлеб пшеничный', 'шт', 32.00, 2500, 2),
(4, 'Яблоки', 'кг', 95.00, 450, 14),
(4, 'Бананы', 'кг', 110.00, 890, 7),
(5, 'Филе трески', 'кг', 450.00, 780, 3),
(5, 'Семга слабосоленая', 'кг', 1200.00, 2020, 5),
(6, 'Картофель', 'кг', 25.00, 770, 30),
(6, 'Помидоры', 'кг', 160.00, 180, 7),
(6, 'Огурцы', 'кг', 150.00, 150, 7),
(7, 'Сок апельсиновый', 'л', 90.00, 450, 30),
(7, 'Чай черный пакетированный', 'шт', 5.00, 10, 365),
(8, 'Сахар', 'кг', 60.00, 3870, 365),
(3, 'Мука пшеничная', 'кг', 50.00, 3640, 180);

```

```

-- Блюда (12)
INSERT INTO menu_item (name, category, selling_price, weight_grams, description)
('Борщ украинский', 'Суп', 180.00, 350, 'Классический борщ с говядиной и сметаной'),
('Куриный суп с лапшой', 'Суп', 120.00, 300, 'Легкий бульон с домашней лапшой'),
('Цезарь с курицей', 'Салат', 220.00, 200, 'Салат с соусом Цезарь, курицей и суха'),
('Оливье', 'Салат', 150.00, 180, 'Традиционный салат Оливье'),
('Стейк из говядины', 'Горячее', 550.00, 200, 'Стейк с картофельным пюре'),
('Рыба запеченная', 'Горячее', 320.00, 180, 'Треска под сливочным соусом'),
('Картофельное пюре', 'Гарнир', 70.00, 200, 'Нежное пюре с молоком'),
('Гречка отварная', 'Гарнир', 50.00, 200, 'Рассыпчатая гречка'),
('Компот ягодный', 'Напиток', 60.00, 250, 'Домашний компот из яблок'),
('Чай с лимоном', 'Напиток', 40.00, 200, 'Черный чай'),
('Шарлотка', 'Десерт', 130.00, 150, 'Яблочный пирог'),
('Банановый смузи', 'Напиток', 160.00, 300, 'Смузи из банана и молока');

-- Состав блюд (menu_item_product) (Создаем связи, примерно 17 записей)
INSERT INTO menu_item_product (menu_item_id, product_id, quantity) VALUES
(1, 1, 0.100), (1, 11, 0.200), (1, 6, 0.050), (1, 12, 0.030), (1, 4, 0.020),
(2, 2, 0.080), (2, 6, 0.040), (2, 17, 0.030),
(3, 2, 0.100), (3, 12, 0.050), (3, 13, 0.030), (3, 4, 0.030),
(5, 1, 0.200), (5, 11, 0.150), (5, 2, 0.010),
(7, 11, 0.200), (7, 2, 0.050),
(8, 11, 0.100),
(12, 2, 0.200), (12, 8, 0.100);

-- Клиенты (20)
INSERT INTO client (first_name, last_name, phone, discount_card, discount_percent)
('Игорь', 'Смирнов', '89150000001', 'DC001', 10),
('Анна', 'Иванова', '89150000002', 'DC002', 5),
('Сергей', 'Петров', '89150000003', 'DC003', 15),
('Екатерина', 'Сидорова', '89150000004', NULL, 0),
('Павел', 'Морозов', '89150000005', 'DC005', 10),
('Юлия', 'Волкова', '89150000006', NULL, 0),
('Максим', 'Зайцев', '89150000007', 'DC007', 5),
('Дарья', 'Медведева', '89150000008', NULL, 0),
('Артем', 'Соколов', '89150000009', 'DC009', 10),
('Анастасия', 'Козлова', '89150000010', NULL, 0),
('Никита', 'Новиков', '89150000011', 'DC011', 20),
('Виктория', 'Лебедева', '89150000012', NULL, 0),
('Денис', 'Егоров', '89150000013', 'DC013', 10),
('Кристина', 'Федорова', '89150000014', NULL, 0),
('Роман', 'Алексеев', '89150000015', 'DC015', 5),
('Полина', 'Борисова', '89150000016', NULL, 0),
('Владислав', 'Григорьев', '89150000017', 'DC017', 15),
('Алина', 'Макарова', '89150000018', NULL, 0),
('Станислав', 'Орлов', '89150000019', 'DC019', 5),
('Евгений', 'Щербаков', '89150000020', NULL, 0);

```



```
-- Заказы (Создадим 25 заказов, чтобы гарантировать общее кол-во строк > 100 с уч
INSERT INTO meal_order (client_id, employee_id, dining_room_id, is_takeaway, paym
(1, 5, 1, false, 'Карта'),
(2, 5, 1, false, 'Наличные'),
(3, 6, 2, false, 'QR-код'),
(null, 7, 3, true, 'Наличные'),
(5, 8, 4, false, 'Карта'),
(6, 9, 4, false, 'Карта'),
(7, 10, 5, false, 'QR-код'),
(8, 11, 5, true, 'Наличные'),
(9, 5, 1, false, 'Карта'),
(10, 6, 2, false, 'Наличные'),
(11, 7, 3, false, 'Карта'),
(12, 8, 4, false, 'Наличные'),
(13, 9, 4, true, 'QR-код'),
(14, 10, 5, false, 'Карта'),
(15, 11, 5, false, 'Карта'),
(1, 12, 1, false, 'Карта'),
(2, 5, 1, false, 'Наличные'),
(null, 6, 2, true, 'Карта'),
(4, 7, 3, false, 'QR-код'),
(6, 8, 4, false, 'Наличные'),
(8, 9, 4, false, 'Карта'),
(10, 10, 5, false, 'Карта'),
(12, 11, 5, true, 'Наличные'),
(14, 12, 1, false, 'QR-код')
```

```
-- Позиции заказов (Создадим по 1-3 позиции на заказ -> > 25 записей)
INSERT INTO order_item (meal_order_id, menu_item_id, quantity, price_per_unit) VA
(1, 1, 2, 180.00), (1, 5, 1, 550.00),
(2, 3, 1, 220.00), (2, 9, 1, 60.00),
(3, 2, 1, 120.00), (3, 7, 1, 70.00),
(4, 12, 2, 160.00),
(5, 4, 1, 150.00), (5, 8, 1, 50.00), (5, 10, 1, 40.00),
(6, 6, 1, 320.00),
(7, 11, 2, 130.00),
(8, 5, 1, 550.00), (8, 9, 1, 60.00),
(9, 1, 1, 180.00),
(10, 3, 1, 220.00), (10, 12, 1, 160.00),
(11, 2, 1, 120.00),
(12, 4, 1, 150.00), (12, 7, 1, 70.00),
(13, 8, 2, 50.00), (13, 6, 1, 320.00),
(14, 11, 1, 130.00), (14, 10, 2, 40.00),
(15, 5, 1, 550.00),
(16, 3, 1, 220.00),
(17, 9, 1, 60.00), (17, 1, 1, 180.00),
(18, 12, 1, 160.00), (18, 7, 1, 70.00),
(19, 6, 1, 320.00),
(20, 8, 1, 50.00), (20, 10, 1, 40.00),
(21, 2, 1, 120.00), (21, 11, 1, 130.00),
(22, 4, 1, 150.00),
(23, 5, 1, 550.00), (23, 9, 1, 60.00)
```

```

-- Остатки на складе (Для двух столовых пример) – около 20 записей
INSERT INTO inventory (dining_room_id, product_id, current_stock, min_stock_level
(1, 1, 15.5, 3.0), (1, 2, 20.0, 5.0), (1, 3, 10.0, 8.0), (1, 11, 50.0, 15.0), (1,
(2, 1, 8.0, 3.0), (2, 2, 12.0, 4.0), (2, 12, 5.0, 2.0), (2, 13, 7.0, 2.0), (2, 9,
(3, 8, 3.0, 1.0), (3, 15, 4.0, 1.0), (3, 14, 8.0, 3.0), (3, 7, 6.0, 2.0),
(4, 17, 25.0, 10.0), (4, 4, 12.0, 5.0), (4, 16, 15.0, 8.0),
(5, 1, 10.0, 4.0), (5, 10, 2.5, 1.0), (5, 17, 18.0, 5.0);

-- Обновляем total_amount в таблице заказов на основе позиций (опционально)
UPDATE meal_order o
SET total_amount = sub.sum_total
FROM (
    SELECT meal_order_id, SUM(quantity * price_per_unit) as sum_total
    FROM order_item
    GROUP BY meal_order_id
) sub
WHERE o.id = sub.meal_order_id;

```

Лабораторная работа №3

```

-- 1. По названию (алфавитный порядок)
SELECT * FROM dining_room ORDER BY name;
-- 2. По вместимости (от малого к большому)
SELECT * FROM dining_room ORDER BY capacity ASC;
-- 3. По времени открытия (кто раньше открывается)
SELECT name, open_time, close_time FROM dining_room ORDER B
-- 4. По времени закрытия (кто позже закрывается – сверху)
SELECT name, open_time, close_time FROM dining_room ORDER B
-- 5. Только открытые столовые, отсортированные по адресу
SELECT * FROM dining_room WHERE is_open = true ORDER BY add

-- 1. По фамилии в алфавитном порядке
SELECT * FROM employee ORDER BY last_name, first_name;

-- 2. По убыванию зарплаты (самые высокооплачиваемые сверху)
SELECT first_name, last_name, position, salary FROM employee ORDER BY salary DESC;

-- 3. По должности, затем по найму (стаж)
SELECT * FROM employee ORDER BY position, hire_date;

-- 4. Сотрудники конкретной столовой (id=1) по имени
SELECT * FROM employee WHERE dining_room_id = 1 ORDER BY first_name;

-- 5. По номеру телефона (естественная сортировка строк)
SELECT first_name, phone FROM employee ORDER BY phone;|

```

```

-- 1. По названию компании
SELECT * FROM supplier ORDER BY company_name;

-- 2. По рейтингу (лучшие сверху)
SELECT company_name, rating FROM supplier ORDER BY rating DESC;

-- 3. По дате начала контракта (давние партнеры сверху)
SELECT company_name, contract_start FROM supplier ORDER BY contract_start ASC;

-- 4. По имени контактного лица
SELECT * FROM supplier ORDER BY contact_person;

-- 5. По убыванию длины названия компании (для интереса)
SELECT company_name, LENGTH(company_name) AS name_len FROM supplier ORDER BY name_

-- 1. По названию продукта
SELECT * FROM product ORDER BY name;

-- 2. По убыванию закупочной цены (дорогие сверху)
SELECT name, purchase_price, unit FROM product ORDER BY purchase_price DESC;

-- 3. По калорийности на единицу (низкокалорийные сначала)
SELECT name, calories_per_unit FROM product ORDER BY calories_per_unit ASC;

-- 4. По сроку годности (скоропортящиеся сверху)
SELECT name, expiration_days FROM product ORDER BY expiration_days ASC;

-- 5. Продукты конкретного поставщика (id=1), отсортированные по единице измерения
SELECT * FROM product WHERE supplier_id = 1 ORDER BY unit, name;

-- 1. По категории, затем по названию
SELECT * FROM menu_item ORDER BY category, name;

-- 2. По цене продажи (дешевые сверху)
SELECT name, selling_price FROM menu_item ORDER BY selling_price ASC;

-- 3. По весу порции (большие порции сверху)
SELECT name, weight_grams FROM menu_item ORDER BY weight_grams DESC;

-- 4. Доступные блюда по убыванию цены
SELECT name, category, selling_price FROM menu_item WHERE is_available = true ORDER BY selling_price DESC;

-- 5. По длине описания (самые подробные сверху)
SELECT name, LENGTH(description) AS desc_len FROM menu_item ORDER BY desc_len DESC

```



```

-- 1. По идентификатору блюда, затем количеству ингредиента
SELECT * FROM menu_item_product ORDER BY menu_item_id, quantity;

-- 2. По убыванию количества (какого ингредиента больше всего на порцию)
SELECT * FROM menu_item_product ORDER BY quantity DESC;

-- 3. По идентификатору продукта
SELECT * FROM menu_item_product ORDER BY product_id;

-- 4. JOIN с названиями – сортировка по названию продукта
SELECT mip.*, p.name AS product_name
FROM menu_item_product mip
JOIN product p ON mip.product_id = p.id
ORDER BY p.name;

-- 5. JOIN с названиями – сортировка по названию блюда
SELECT mip.*, mi.name AS menu_name
FROM menu_item_product mip
JOIN menu_item mi ON mip.menu_item_id = mi.id
ORDER BY mi.name;

-- 1. По фамилии и имени
SELECT * FROM client ORDER BY last_name, first_name;

-- 2. По проценту скидки (привилегированные сверху)
SELECT first_name, last_name, discount_percent FROM client ORDER BY discount_percent DESC;

-- 3. По дате регистрации (новые клиенты сверху)
SELECT * FROM client ORDER BY registered_at DESC;

-- 4. Только клиенты с днем рождения в этом году, отсортированные по дате
SELECT first_name, last_name, birthday FROM client
WHERE birthday IS NOT NULL
ORDER BY EXTRACT(MONTH FROM birthday), EXTRACT(DAY FROM birthday);

-- 5. По номеру дисконтной карты (с пустыми в конце)
SELECT * FROM client ORDER BY discount_card NULLS LAST;

-- 1. По дате заказа (свежие сверху)
SELECT * FROM meal_order ORDER BY order_date DESC;

-- 2. По сумме заказа (дорогие сверху)
SELECT * FROM meal_order ORDER BY total_amount DESC;

-- 3. По методу оплаты, затем по дате
SELECT * FROM meal_order ORDER BY payment_method, order_date;

-- 4. Только заказы на вынос, отсортированные по id столовой
SELECT * FROM meal_order WHERE is_takeaway = true ORDER BY dining_room_id, order_date;

-- 5. По идентификатору клиента, пустые (гости без карты) в конце
SELECT * FROM meal_order ORDER BY client_id NULLS LAST;

```

```

-- 1. По номеру заказа, затем цене за единицу
SELECT * FROM order_item ORDER BY meal_order_id, price_per_unit;

-- 2. По убыванию количества (больше всего порций в позиции)
SELECT * FROM order_item ORDER BY quantity DESC;

-- 3. По общей стоимости позиции (quantity * price_per_unit) – дорогие сверху
SELECT *, (quantity * price_per_unit) AS total_line FROM order_item ORDER BY total_line DESC;

-- 4. По идентификатору блюда
SELECT * FROM order_item ORDER BY menu_item_id;

-- 5. JOIN для вывода названия блюда, сортировка по названию
SELECT oi.*, mi.name
FROM order_item oi
JOIN menu_item mi ON oi.menu_item_id = mi.id
ORDER BY mi.name;

-- 1. По столовой, затем продукту
SELECT * FROM inventory ORDER BY dining_room_id, product_id;

-- 2. По текущему остатку (заканчивающиеся сверху)
SELECT * FROM inventory ORDER BY current_stock ASC;

-- 3. По убыванию разницы между текущим и минимальным запасом (излишки)
SELECT *, (current_stock - min_stock_level) AS surplus FROM inventory ORDER BY surplus DESC;

-- 4. Продукты с остатком ниже минимального уровня (дефицит), по возрастанию остатка
SELECT * FROM inventory WHERE current_stock < min_stock_level ORDER BY current_stock ASC;

-- 5. JOIN с названием продукта, сортировка по названию продукта
SELECT i.*, p.name AS product_name
FROM inventory i
JOIN product p ON i.product_id = p.id
ORDER BY p.name;

```

Лабораторная работа 4

```

CREATE TABLE supply (
    id SERIAL PRIMARY KEY,
    supplier_id INT NOT NULL,
    product_id INT NOT NULL,
    dining_room_id INT NOT NULL,
    supply_date DATE NOT NULL DEFAULT CURRENT_DATE,
    quantity NUMERIC(10,3) NOT NULL CHECK (quantity > 0),
    price_per_unit NUMERIC(10,2) NOT NULL CHECK (price_per_unit > 0),
    total_cost NUMERIC(12,2) GENERATED ALWAYS AS (quantity * price_per_unit) STORED,
    invoice_number VARCHAR(50) UNIQUE,
    CONSTRAINT fk_supply_supplier FOREIGN KEY (supplier_id) REFERENCES supplier(id)
    CONSTRAINT fk_supply_product FOREIGN KEY (product_id) REFERENCES product(id) ON DELETE CASCADE
    CONSTRAINT fk_supply_dining FOREIGN KEY (dining_room_id) REFERENCES dining_room(id) ON DELETE CASCADE
);

```

```

-- supplier: добавляем ответственного сотрудника
ALTER TABLE supplier ADD COLUMN responsible_employee_id INT;
ALTER TABLE supplier ADD CONSTRAINT fk_supplier_employee
    FOREIGN KEY (responsible_employee_id) REFERENCES employee(id) ON DELETE SET NU

-- menu_item: добавляем создателя блюда (шеф-повара)
ALTER TABLE menu_item ADD COLUMN created_by_employee_id INT;
ALTER TABLE menu_item ADD CONSTRAINT fk_menu_creator
    FOREIGN KEY (created_by_employee_id) REFERENCES employee(id) ON DELETE SET NUL

-- client: добавляем любимую столовую
ALTER TABLE client ADD COLUMN favorite_dining_room_id INT;
ALTER TABLE client ADD CONSTRAINT fk_client_dining
    FOREIGN KEY (favorite_dining_room_id) REFERENCES dining_room(id) ON DELETE SET

-- supplier
UPDATE supplier SET responsible_employee_id = 12 WHERE id = 1;
UPDATE supplier SET responsible_employee_id = 5 WHERE id = 2;
UPDATE supplier SET responsible_employee_id = 8 WHERE id = 3;
UPDATE supplier SET responsible_employee_id = 11 WHERE id = 4;
UPDATE supplier SET responsible_employee_id = 2 WHERE id = 5;
UPDATE supplier SET responsible_employee_id = 10 WHERE id = 6;
UPDATE supplier SET responsible_employee_id = 6 WHERE id = 7;
UPDATE supplier SET responsible_employee_id = 3 WHERE id = 8;

-- menu_item
UPDATE menu_item SET created_by_employee_id = 1 WHERE category IN ('Суп', 'Горяче
UPDATE menu_item SET created_by_employee_id = 4 WHERE category = 'Салат';
UPDATE menu_item SET created_by_employee_id = 10 WHERE category = 'Десерт';
UPDATE menu_item SET created_by_employee_id = 11 WHERE category = 'Напиток';
UPDATE menu_item SET created_by_employee_id = 6 WHERE category = 'Гарнир';

-- client
UPDATE client SET favorite_dining_room_id = 1 WHERE id % 5 = 1;
UPDATE client SET favorite_dining_room_id = 2 WHERE id % 5 = 2;
UPDATE client SET favorite_dining_room_id = 3 WHERE id % 5 = 3;
UPDATE client SET favorite_dining_room_id = 4 WHERE id % 5 = 4;
UPDATE client SET favorite_dining_room_id = 5 WHERE id % 5 = 0;|

```

```

INSERT INTO supply (supplier_id, product_id, dining_room_id, supply_date, quantity,
(1, 1, 1, '2026-01-10', 25.0, 645.00, 'INV-001'),
(1, 2, 1, '2026-01-10', 30.0, 278.00, 'INV-002'),
(2, 3, 1, '2026-01-12', 50.0, 64.00, 'INV-003'),
(2, 4, 1, '2026-01-12', 15.0, 218.00, 'INV-004'),
(3, 5, 2, '2026-01-15', 40.0, 34.50, 'INV-005'),
(3, 6, 2, '2026-01-15', 35.0, 31.50, 'INV-006'),
(4, 7, 2, '2026-01-18', 20.0, 94.00, 'INV-007'),
(4, 8, 3, '2026-01-18', 15.0, 108.00, 'INV-008'),
(5, 9, 3, '2026-01-20', 10.0, 448.00, 'INV-009'),
(5, 10, 3, '2026-01-20', 8.0, 1190.00, 'INV-010'),
(6, 11, 4, '2026-02-01', 100.0, 24.50, 'INV-011'),
(6, 12, 4, '2026-02-01', 30.0, 158.00, 'INV-012'),
(6, 13, 4, '2026-02-01', 25.0, 148.00, 'INV-013'),
(7, 14, 5, '2026-02-05', 40.0, 88.00, 'INV-014'),
(7, 15, 5, '2026-02-05', 200.0, 4.80, 'INV-015'),
(8, 16, 1, '2026-02-10', 50.0, 59.00, 'INV-016'),
(3, 17, 5, '2026-02-12', 60.0, 49.00, 'INV-017'),
(1, 1, 3, '2026-02-15', 15.0, 648.00, 'INV-018'),
(2, 3, 4, '2026-02-18', 30.0, 65.00, 'INV-019'),
(6, 11, 2, '2026-02-20', 45.0, 25.00, 'INV-020');

```

-- 1. Общее количество столовых

```
SELECT COUNT(*) AS total_dining_rooms FROM dining_room;
```

-- 2. Средняя вместимость

```
SELECT ROUND(AVG(capacity), 2) AS avg_capacity FROM dining_room;
```

-- 3. Максимальная и минимальная вместимость

```
SELECT MAX(capacity) AS max_cap, MIN(capacity) AS min_cap FROM dining_room;
```

-- 4. Суммарная вместимость всех столовых

```
SELECT SUM(capacity) AS total_capacity FROM dining_room;
```

-- 5. Количество открытых и закрытых столовых

```
SELECT is_open, COUNT(*) AS cnt FROM dining_room GROUP BY is_open;
```

-- 1. Средняя зарплата

```
SELECT ROUND(AVG(salary), 2) AS avg_salary FROM employee;
```

-- 2. Фонд оплаты труда по должностям

```
SELECT position, SUM(salary) AS position_fund, COUNT(*) AS cnt
FROM employee GROUP BY position ORDER BY position_fund DESC;
```

-- 3. Минимальная и максимальная зарплата по столовым

```
SELECT dining_room_id, MIN(salary) AS min_sal, MAX(salary) AS max_sal
FROM employee GROUP BY dining_room_id;
```

-- 4. Количество сотрудников в каждой столовой

```
SELECT dining_room_id, COUNT(*) AS emp_count FROM employee GROUP BY dining_room_id;
```

-- 5. Стандартное отклонение зарплаты

```
SELECT ROUND(STDDEV(salary), 2) AS salary_stddev FROM employee;
```

```

-- 1. Средняя зарплата
SELECT ROUND(AVG(salary), 2) AS avg_salary FROM employee;

-- 2. Фонд оплаты труда по должностям
SELECT position, SUM(salary) AS position_fund, COUNT(*) AS cnt
FROM employee GROUP BY position ORDER BY position_fund DESC;

-- 3. Минимальная и максимальная зарплата по столовым
SELECT dining_room_id, MIN(salary) AS min_sal, MAX(salary) AS max_sal
FROM employee GROUP BY dining_room_id;

-- 4. Количество сотрудников в каждой столовой
SELECT dining_room_id, COUNT(*) AS emp_count FROM employee GROUP BY dining_room_id;

-- 5. Стандартное отклонение зарплаты
SELECT ROUND(STDDEV(salary), 2) AS salary_stddev FROM employee;

-- 1. Средний рейтинг поставщиков
SELECT ROUND(AVG(rating), 2) AS avg_rating FROM supplier;

-- 2. Количество поставщиков с рейтингом выше 8
SELECT COUNT(*) AS top_suppliers FROM supplier WHERE rating > 8;

-- 3. Минимальный и максимальный рейтинг
SELECT MIN(rating) AS min_r, MAX(rating) AS max_r FROM supplier;

-- 4. Сумма рейтингов (условная)
SELECT SUM(rating) AS total_rating FROM supplier;

-- 5. Количество поставщиков, у которых есть email
SELECT COUNT(*) AS with_email FROM supplier WHERE email IS NOT NULL;

- 1. Средняя закупочная цена
SELECT ROUND(AVG(purchase_price), 2) AS avg_price FROM product;

- 2. Суммарная калорийность всех продуктов (на единицу)
SELECT SUM(calories_per_unit) AS total_cal FROM product;

- 3. Максимальный срок годности
SELECT MAX(expiration_days) AS max_exp FROM product;

- 4. Количество продуктов по единицам измерения
SELECT unit, COUNT(*) AS cnt FROM product GROUP BY unit;

- 5. Минимальная цена среди алкогольных/безалкогольных
SELECT is_alcohol, MIN(purchase_price) AS min_price FROM product GROUP BY is_alcohol

```



```

-- 1. Средняя цена продажи
SELECT ROUND(AVG(selling_price), 2) AS avg_price FROM menu_item;

-- 2. Суммарный вес всех уникальных блюд
SELECT SUM(weight_grams) AS total_weight FROM menu_item;

-- 3. Максимальная и минимальная цена по категориям
SELECT category, MAX(selling_price) AS max_p, MIN(selling_price) AS min_p
FROM menu_item GROUP BY category;

-- 4. Количество доступных блюд
SELECT COUNT(*) AS available_items FROM menu_item WHERE is_available = true;

-- 5. Средний вес порции по категориям
SELECT category, ROUND(AVG(weight_grams), 0) AS avg_weight
FROM menu_item GROUP BY category;

-- 1. Суммарное количество всех ингредиентов во всех блюдах
SELECT SUM(quantity) AS total_qty FROM menu_item_product;

-- 2. Среднее количество ингредиента на порцию
SELECT ROUND(AVG(quantity), 3) AS avg_qty FROM menu_item_product;

-- 3. Сколько всего ингредиентов в каждом блюде
SELECT menu_item_id, COUNT(*) AS ingr_count
FROM menu_item_product GROUP BY menu_item_id;

-- 4. Максимальное количество одного ингредиента в блюде
SELECT MAX(quantity) AS max_qty FROM menu_item_product;

-- 5. Суммарный вес ингредиентов на порцию каждого блюда
SELECT menu_item_id, SUM(quantity) AS total_per_portion
FROM menu_item_product GROUP BY menu_item_id;

-- 1. Средний процент скидки
SELECT ROUND(AVG(discount_percent), 2) AS avg_discount FROM client;

-- 2. Максимальная скидка
SELECT MAX(discount_percent) AS max_disc FROM client;

-- 3. Количество клиентов с картой лояльности
SELECT COUNT(*) AS with_card FROM client WHERE discount_card IS NOT NULL;

-- 4. Суммарный скидочный процент (условно)
SELECT SUM(discount_percent) AS total_disc FROM client;

-- 5. Количество клиентов по любимым столовым
SELECT favorite_dining_room_id, COUNT(*) AS cnt
FROM client GROUP BY favorite_dining_room_id;

```

```

-- 1. Общая выручка
SELECT SUM(total_amount) AS total_revenue FROM meal_order;

-- 2. Средний чек
SELECT ROUND(AVG(total_amount), 2) AS avg_check FROM meal_order;

-- 3. Количество заказов по способу оплаты
SELECT payment_method, COUNT(*) AS cnt FROM meal_order GROUP BY payment_method;

-- 4. Минимальный и максимальный чек
SELECT MIN(total_amount) AS min_check, MAX(total_amount) AS max_check FROM meal_order;

-- 5. Выручка по дням
SELECT DATE(order_date) AS day, SUM(total_amount) AS daily_rev
FROM meal_order GROUP BY DATE(order_date) ORDER BY day;

```

```

-- 1. Общее количество проданных порций
SELECT SUM(quantity) AS total_portions FROM order_item;

-- 2. Средняя цена позиции
SELECT ROUND(AVG(price_per_unit), 2) AS avg_price FROM order_item;

-- 3. Суммарная выручка по блюдам
SELECT menu_item_id, SUM(quantity * price_per_unit) AS item_revenue
FROM order_item GROUP BY menu_item_id ORDER BY item_revenue DESC;

-- 4. Максимальное количество единиц в одной позиции
SELECT MAX(quantity) AS max_qty FROM order_item;

-- 5. Количество позиций в каждом заказе
SELECT meal_order_id, COUNT(*) AS items_count
FROM order_item GROUP BY meal_order_id;

```

```

-- 1. Общий остаток по всем складам
SELECT SUM(current_stock) AS total_stock FROM inventory;

-- 2. Средний запас продукта
SELECT ROUND(AVG(current_stock), 2) AS avg_stock FROM inventory;

-- 3. Минимальный остаток по продуктам
SELECT product_id, MIN(current_stock) AS min_stock
FROM inventory GROUP BY product_id;

-- 4. Количество позиций с дефицитом
SELECT COUNT(*) AS deficit_items FROM inventory WHERE current_stock < min_stock_level;

-- 5. Суммарный страховой запас (min_stock_level)
SELECT SUM(min_stock_level) AS total_min FROM inventory;

```

```
-- 1. Общая сумма затрат на поставки
SELECT SUM(total_cost) AS total_spent FROM supply;

-- 2. Средняя цена за единицу в поставках
SELECT ROUND(AVG(price_per_unit), 2) AS avg_unit_price FROM supply;

-- 3. Максимальная по объёму поставка
SELECT MAX(quantity) AS max_qty FROM supply;

-- 4. Количество поставок по поставщикам
SELECT supplier_id, COUNT(*) AS deliveries FROM supply GROUP BY supplier_id;

-- 5. Суммарный объём поставок по продуктам
SELECT product_id, SUM(quantity) AS total_qty FROM supply GROUP BY product_id;
```

